Deep Learning | Multi-Task Learning | Medical Imaging

CNN-Based COVID-19 Diagnostic Support System

Multi-Task Transfer Learning on Chest X-Ray Images

Joint Classification & Lung Segmentation with Explainable AI

Mohammed Abdulai
Student

Peter Mvuma
Student

Group 1

Problem Statement

The challenges of accurate chest X-ray diagnosis persist, affecting patients' outcomes. Misdiagnosis can lead to delayed treatment and increased healthcare costs.

Our goal is to leverage multi-task transfer learning to enhance diagnosis accuracy and efficiency in identifying various conditions from chest X-rays.

Imaging findings can overlap across respiratory infections, making classification challenging.

AI and deep learning are improving chest X-ray interpretation for respiratory disease diagnosis while reducing manual review burden.

Project Objectives

1

Multiclass Classification

Classify chest X-rays into 4 clinically relevant categories:

-> COVID-19
-> Lung Opacity
-> Viral Pneumonia
-> Normal

2

Multi-Task Classification

Evaluate whether multi-task learning improves classification performance over single-task CNN baselines (EfficientNet, ResNet, DenseNet).

3

Joint Lung Segmentation

Simultaneously predict pixel-level lung segmentation masks using a shared DenseNet encoder, which enabled anatomically guided learning.

4

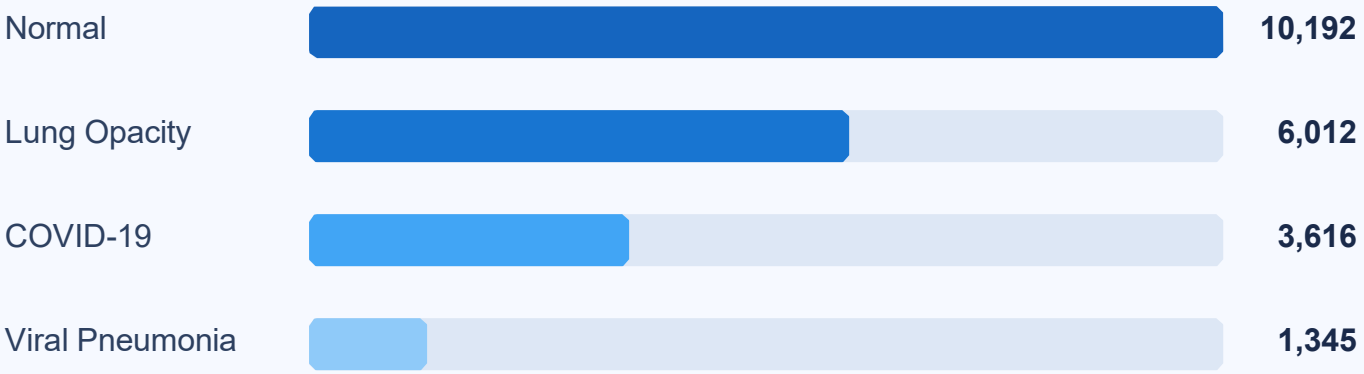
Explainable AI

Apply Grad-CAM visualisations to highlight lung regions influencing predictions, increasing transparency and clinical interpretability.

DATA

Dataset Overview — Kaggle COVID-19 Radiography Database

CLASS DISTRIBUTION (21,165 TOTAL IMAGES)



21,165
Total Chest X-rays

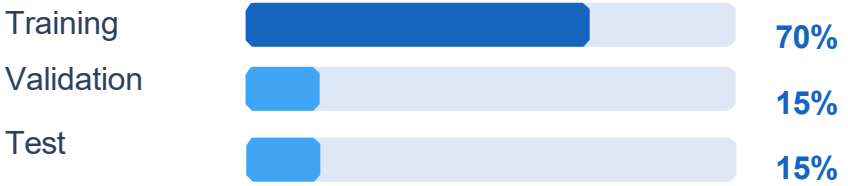
21,165
Paired Lung Masks

224px
Input Resolution

PREPROCESSING PIPELINE

1. Resize all images to 224 x 224 pixels
2. Normalize pixel values to [0, 1] — convert to RGB format
3. Data augmentation: horizontal flip + minor rotations
4. Integrity check: removed corrupted or duplicate images

DATASET SPLIT (STRATIFIED)



Project Data source Kaggle https://www.kaggle.com/datasets/tawsifurrahman/covid19-radiography-database

Image Source | Kaggle COVID-19 Radiography Database - COVID-19 Radiography Database

Handling Dataset Imbalance

IMBALANCE OBSERVATION

- The dataset is imbalanced across classes
- The normal class dominates the dataset
- Viral Pneumonia is highly underrepresented
- Risk of model bias toward majority classes

APPROACH USED

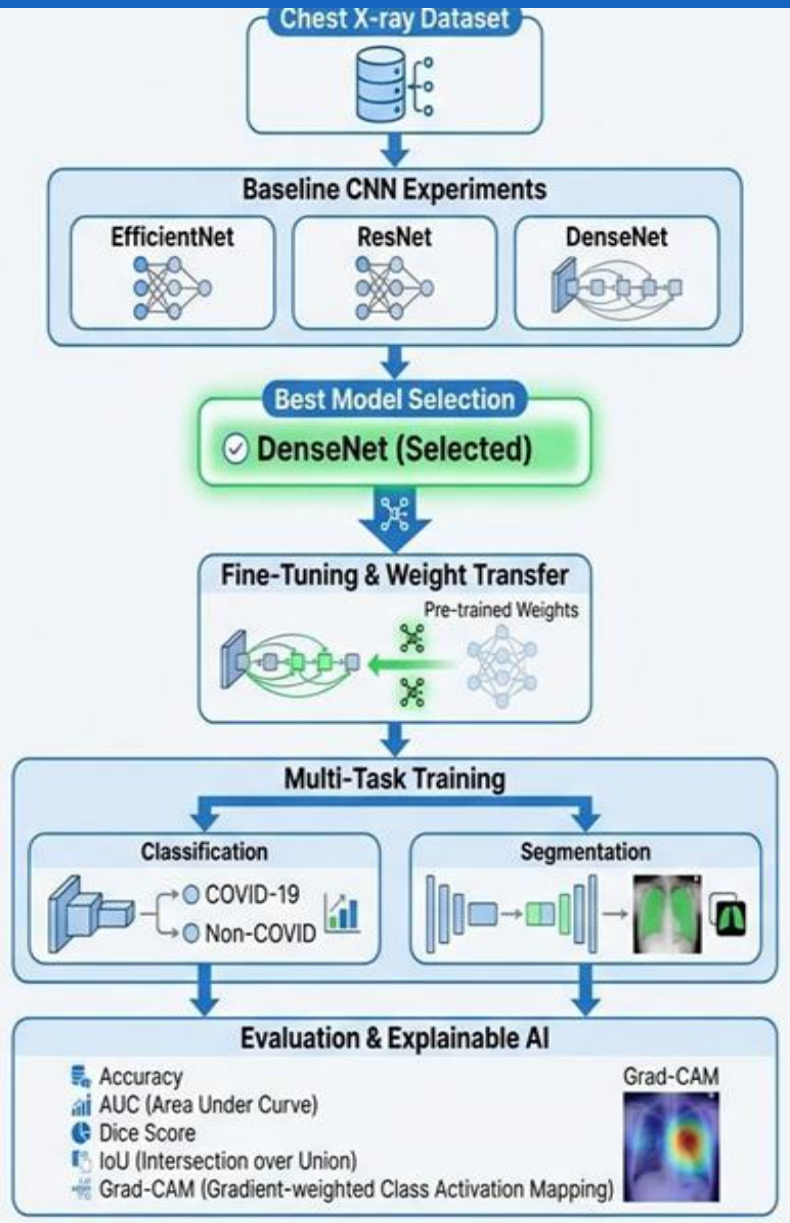
Stratified Train/Validation/Test Split

- Done to preserve class proportions

Data Augmentation Applied

- Horizontal flipping & small rotations
- No class weighting or oversampling –hence no explicit rebalancing techniques applied

Full project flow chart



Graphics created by scispace

Progressive Methodology Implementation of the Pipeline

01 Dataset
21,165 chest X-rays + lung masks (Kaggle)

→

02 Preprocessing
Resize 224x224, normalize, augment, stratified split

→

03 Baseline CNNs
EfficientNet, ResNet, DenseNet — ImageNet weights

→

04 Best Model
DenseNet selected — highest

→

05 Fine-Tuning
Unfreeze last 40 layers

→

06 Weight Transfer
Transfer fine-tuned DenseNet weights to multi-task model

→

07 Multi-Task
Joint classification + segmentation

KEY INNOVATION

Instead of initialising the multi-task model with ImageNet weights, the best weights from the fine-tuned DenseNet were transferred. This provided the domain-adapted features, faster convergence and improved stability for both tasks.

Baseline DenseNet Model

Baseline Model — DenseNet121

```python
def build_densenet_baseline(num_classes=NUM_CLASSES):
    inputs = keras.Input(shape=(224,224,3))

    base = keras.applications.DenseNet121(
        include_top=False,
        weights="imagenet",
        input_tensor=inputs
    )
    base.trainable = False  # baseline = frozen

    x = layers.GlobalAveragePooling2D()(base.output)
    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(num_classes, activation="softmax")(x)

    model = keras.Model(inputs, outputs, name="DenseNet_Baseline")
    return model

baseline_model =  build_densenet_baseline(NUM_CLASSES)
baseline_model.summary()
```

```
Total params: 7,041,604 (26.86 MB)

Trainable params: 4,100 (16.02 KB)

Non-trainable params: 7,037,504 (26.85 MB)
```

```python
# Compile Model
baseline_model.compile(
```

Hyperparameters	Value
Optimiser	Adam
Initial LR (frozen)	1e-4
Loss function	Categorical cross entropy
Dropout Rate	0.3
Frozen Layers	All
Batch Size	32
Epochs	30 (Callback early stopping)

Fine-Tuned Baseline DenseNet Model

Fine-Tuning Baseline Model

```
# Unfreeze Last 40 Layers

for layer in baseline_model.layers[-40:]:
    layer.trainable = True

for layer in baseline_model.layers[:-40]:
    layer.trainable = False
```

```
# Recompile

baseline_model.compile(
    optimizer=keras.optimizers.Adam(1e-5),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
# Retrain
history_finetune = baseline_model.fit(
    train_ds_base,
    validation_data=val_ds_base,
    epochs=30,
    callbacks=[
        keras.callbacks.EarlyStopping(
            monitor="val_accuracy",
            patience=3,
            restore_best_weights=True,
            mode="max"
        )
    ]
)
```

```
Epoch 1/30
463/463 ━━━━━━━━━━━━━━━━━━━━ 1388s 3s/step - accuracy: 0.6654 - loss: 1.0009 - val_accuracy: 0.8104 - val_loss: 0.5186
Epoch 2/30
463/463 ━━━━━━━━━━━━━━━━━━━━ 1365s 3s/step - accuracy: 0.7714 - loss: 0.6436 - val_accuracy: 0.8233 - val_loss: 0.4816
```

Hyperparameters	Value
Optimiser	Adam
Initial LR (frozen)	1e-5
Loss function	Categorical cross entropy
Dropout Rate	0.3
Unfrozen Layers	Last 40
Batch Size	32
Epochs	30 (Callback early stopping)

Note: Showing part of the DenseNet baseline and for the rest of the baseline model, codes are uploaded on GitHub.

RESULTS

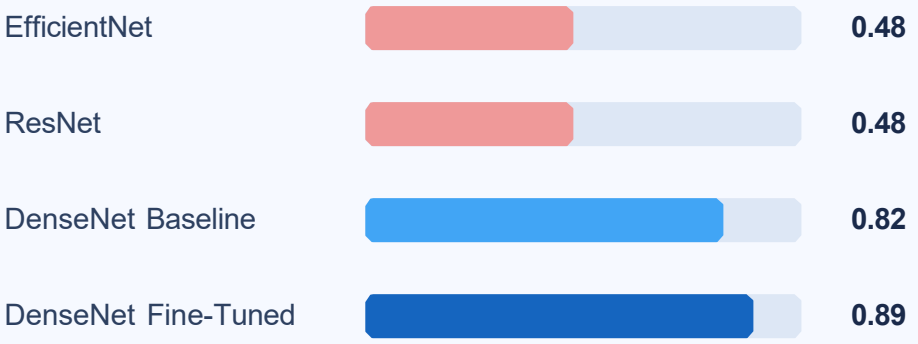
Baseline Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	AUC
EfficientNet (Baseline)	0.48	0.12	0.25	0.16	0.72
ResNet (Baseline)	0.48	0.12	0.25	0.16	0.72
DenseNet (Baseline)	0.82	0.80	0.79	0.79	0.95
DenseNet (Fine-Tuned)	**0.89**	**0.92**	**0.86**	**0.88**	**0.98**

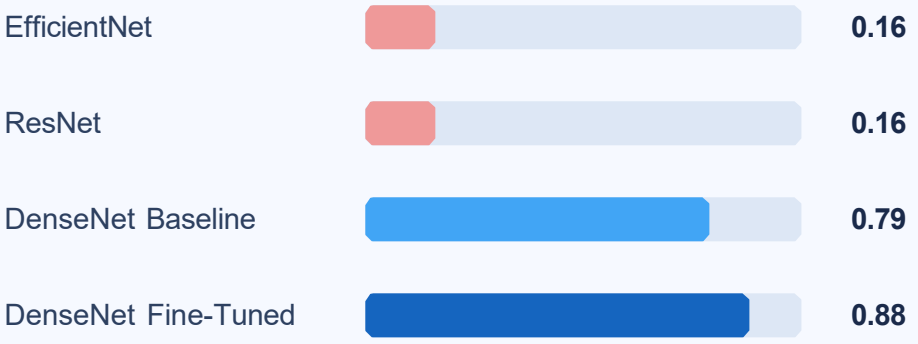
Key Insight

EfficientNet and ResNet failed to learn meaningful class distinctions (F1 score = 0.16). DenseNet's dense connectivity and feature reuse gave a massive advantage, 0.82 accuracy vs. 0.48 for competing architectures.

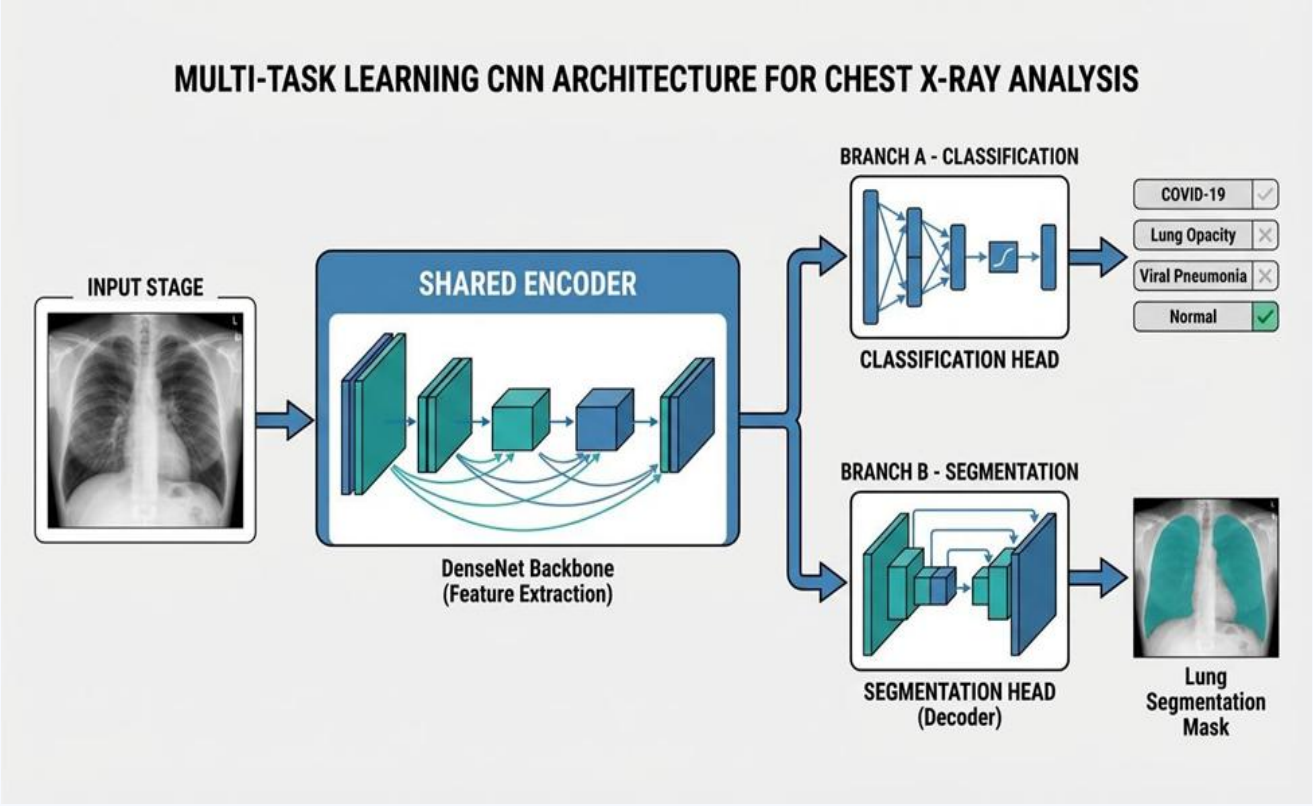
ACCURACY COMPARISON



F1-SCORE COMPARISON



Multi-Task DenseNet Architecture



Shared Encoder

DenseNet121 backbone pre-trained on ImageNet, fine-tuned on chest X-rays. Dense connectivity enabled feature reuse across all layers.

Classification Head

Global Average Pooling + Dropout (0.3) + Dense (4, softmax). Outputs probabilities for 4 disease classes.

Segmentation Head

U-Net style decoder with skip connections. Outputs binary lung mask at 224x224 resolution (sigmoid activation).

Phase 1
Train classification head, encoder frozen

Phase 1.5
Joint fine-tuning of both heads

Weight Transfer from the Fine-Tuned Model

```python
# Load Baseline Fine-Tuned Weights into Backbone

baseline_model = build_densenet_baseline(NUM_CLASSES)

baseline_model.load_weights(
    r"C:\Users\amysh\Desktop\AI_Project\DenseNet\checkpoints\densenet_baseline_finetuned.weights.h5", skip_mismatch=True
)
```

```python
# Build Multi-Task DenseNet
multitask_model, backbone = build_multitask_densenet(
    img_shape=(224,224,3),
    num_classes=NUM_CLASSES
)
```

```python
# Transfer weights into multitask backbone
baseline_weights = baseline_model.get_weights()
```

```python
# Transfer weights into multitask backbone
baseline_weights = baseline_model.get_weights()
```

```python
# Frozen Backbone
backbone.trainable = False

# Freeze ENTIRE DenseNet backbone

for layer in backbone.layers:
    layer.trainable = False

# Double check number of trainable layers
print("Backbone trainable:", backbone.trainable)
print("Trainable layers:", sum([l.trainable for l in backbone.layers]))

# Check first backbone weight (should NOT be random)
print("Backbone weight mean:",
      np.mean(backbone.get_weights()[0]))

multitask_model.summary()
```

```
Backbone trainable: False
Trainable layers: 0
Backbone weight mean: -0.0003748805
```

TRAINING

Training Configuration for Mult-Task Training Phase 1

Train Phase 1

```
[ ]: # Compile (Balanced Loss)

multitask_model.compile(
    optimizer=keras.optimizers.Adam(1e-4),
    loss={
        "classification": "categorical_crossentropy",
        "segmentation": "binary_crossentropy"
    },
    loss_weights={
        "classification": 1.0,
        "segmentation": 0.1   # keep segmentation controlled
    },
    metrics={
        "classification": ["accuracy"]
    }
)

phase1_path = r"C:\Users\amysh\Desktop\AI_Project\DenseNet\checkpoints\multitask_phase1_best.weights.h5" # checkpoint to save best weights

history_phase1 = multitask_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=15,
    callbacks=[
        keras.callbacks.ModelCheckpoint(
            filepath=phase1_path,
            monitor="val_classification_accuracy",
            save_best_only=True,
            save_weights_only=True,
            mode="max",
            verbose=1
        ),
        keras.callbacks.EarlyStopping(
            monitor="val_classification_accuracy",
            patience=3,
            restore_best_weights=True,
            mode="max"
        )
    ]
)
```

```
Epoch 1/15
463/463 ━━━━━━━━━━━━━━━━━━━━ 0s 3s/step - classification_accuracy: 0.6171 - classification_loss: 1.0533 - loss: 1.0862 - segmentation_loss: 0.3288
Epoch 1: val_classification_accuracy improved from None to 0.81165, saving model to C:\Users\amysh\Desktop\AI_Project\DenseNet\checkpoints\multitask_
phase1_best.weights.h5
463/463 ━━━━━━━━━━━━━━━━━━━━ 1782s 4s/step - classification_accuracy: 0.7087 - classification_loss: 0.8236 - loss: 0.8492 - segmentation_loss: 0.2562
```

Hyperparameters	Value
Optimiser	Adam
Initial LR (frozen)	1e-4
Segmentation weight	0.1
Dropout Rate	0.3
frozen Layers	Entire layers
Batch Size	32
Epochs	15 (early stop)

Training Configuration for Mult-Task Training Phase 1.5

PHASE 1.5 — FINE-TUNE (λ = 0.1)

```
[ ]: # Unfreeze last N layers of DenseNet backbone
N = 40

for layer in backbone.layers[:-N]:
    layer.trainable = False
for layer in backbone.layers[-N:]:
    layer.trainable = True

# (Optional but recommended) Keep BatchNorm frozen for stability
for layer in backbone.layers:
    if isinstance(layer, tf.keras.layers.BatchNormalization):
        layer.trainable = False

print("Trainable backbone layers:",
      sum([l.trainable for l in backbone.layers]))

# Recompile (required after changing trainable flags)
multitask_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss={
        "classification": "categorical_crossentropy",
        "segmentation": "binary_crossentropy"
    },
    loss_weights={
        "classification": 1.0,
        "segmentation": 0.1
    },
    metrics={"classification": ["accuracy"]}
)

# 3) Train + save best weights
phase1_5_path = r"C:\Users\amysh\Desktop\AI_Project\DenseNet\checkpoints\multitask_phase1_5_best.weights.h5"

history_phase1_5 = multitask_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20,
    callbacks=[
        keras.callbacks.ModelCheckpoint(
            filepath=phase1_5_path,
            monitor="val_classification_accuracy",
            save_best_only=True,
            save_weights_only=True,
            mode="max",
            verbose=1
        ),
        keras.callbacks.EarlyStopping(
            monitor="val_classification_accuracy",
            patience=3,
            restore_best_weights=True,
            mode="max"
        ),
        keras.callbacks.ReduceLROnPlateau(
            monitor="val_classification_loss",
            factor=0.5,
            patience=2,
            min_lr=1e-7,
            verbose=1,
            mode="min"
        )
    ]
)
```

```
Trainable backbone layers: 29
Epoch 1/20
463/463 ━━━━━━━━━━━━━━━━━━━━ 0s 3s/step - classification_accuracy: 0.8309 - classification_loss: 0.4663 - loss: 0.4833 - segmentation_loss: 0.1708
```

Hyperparameters	Value
Optimiser	Adam
Initial LR (frozen)	1e-4
Segmentation weight	0.1
Dropout Rate	0.3
Unfrozen Layers	Last 40
Batch Size	32
Epochs	20 (early stopping)

RESULTS

Multi-Task Learning Results

0.84	0.83	0.92	0.986	0.89	0.80
Phase 1 Accuracy	Phase 1 F1-Score	Phase 1.5 Accuracy	AUC Score	Dice Score	IoU Score
Classification head only	Macro-averaged	Joint fine-tuning	Phase 1.5 final	Segmentation	Segmentation

CLASSIFICATION METRICS (Phase 1.5)

Accuracy	0.92
F1-Score	0.91
AUC	0.986

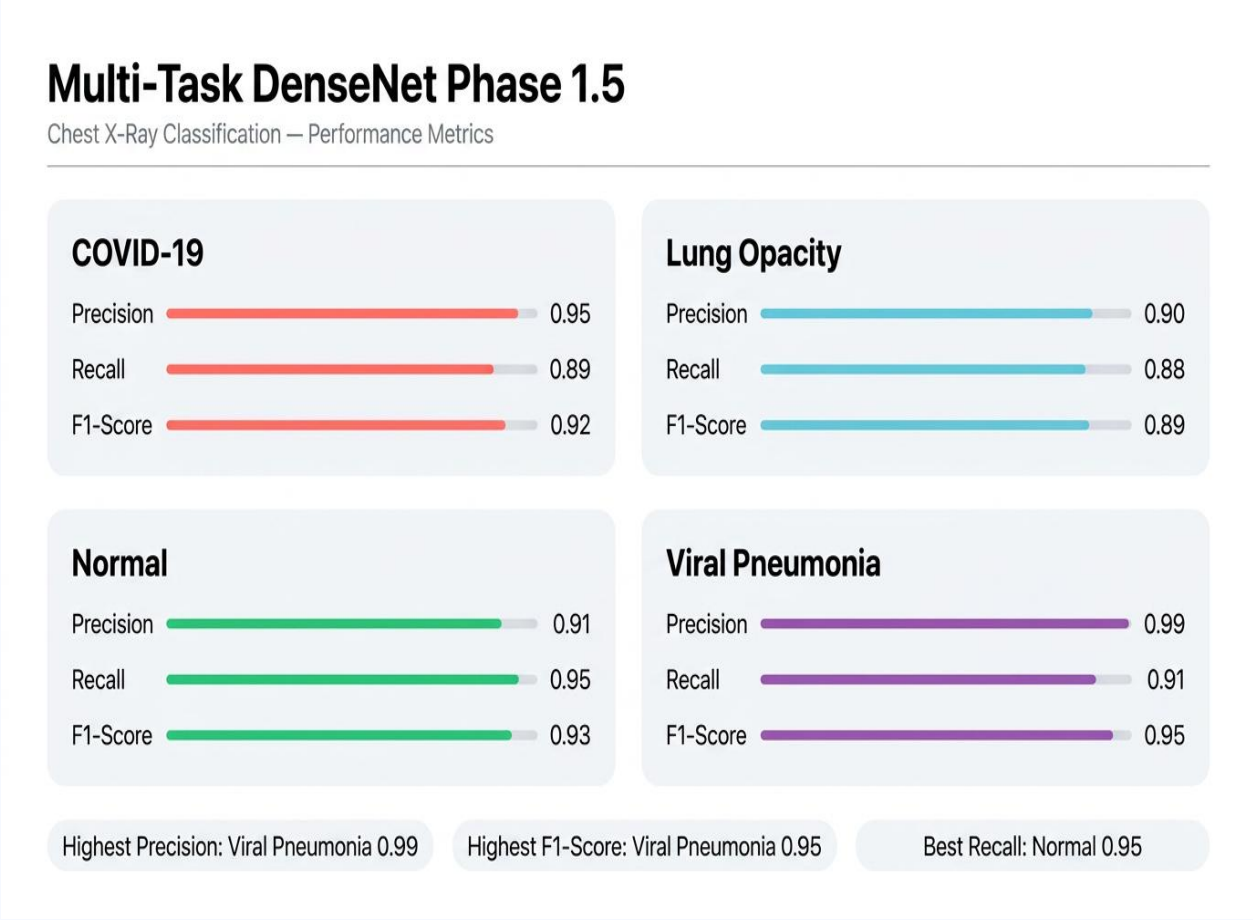
SEGMENTATION METRICS

Dice Score	0.89
IoU Score	0.80

Phase 1: 0.84 acc
Classification head trained, encoder frozen

Phase 1.5: 0.92 acc
Joint fine-tuning of both heads — best model

Model Performance Metrics for Specific Classes

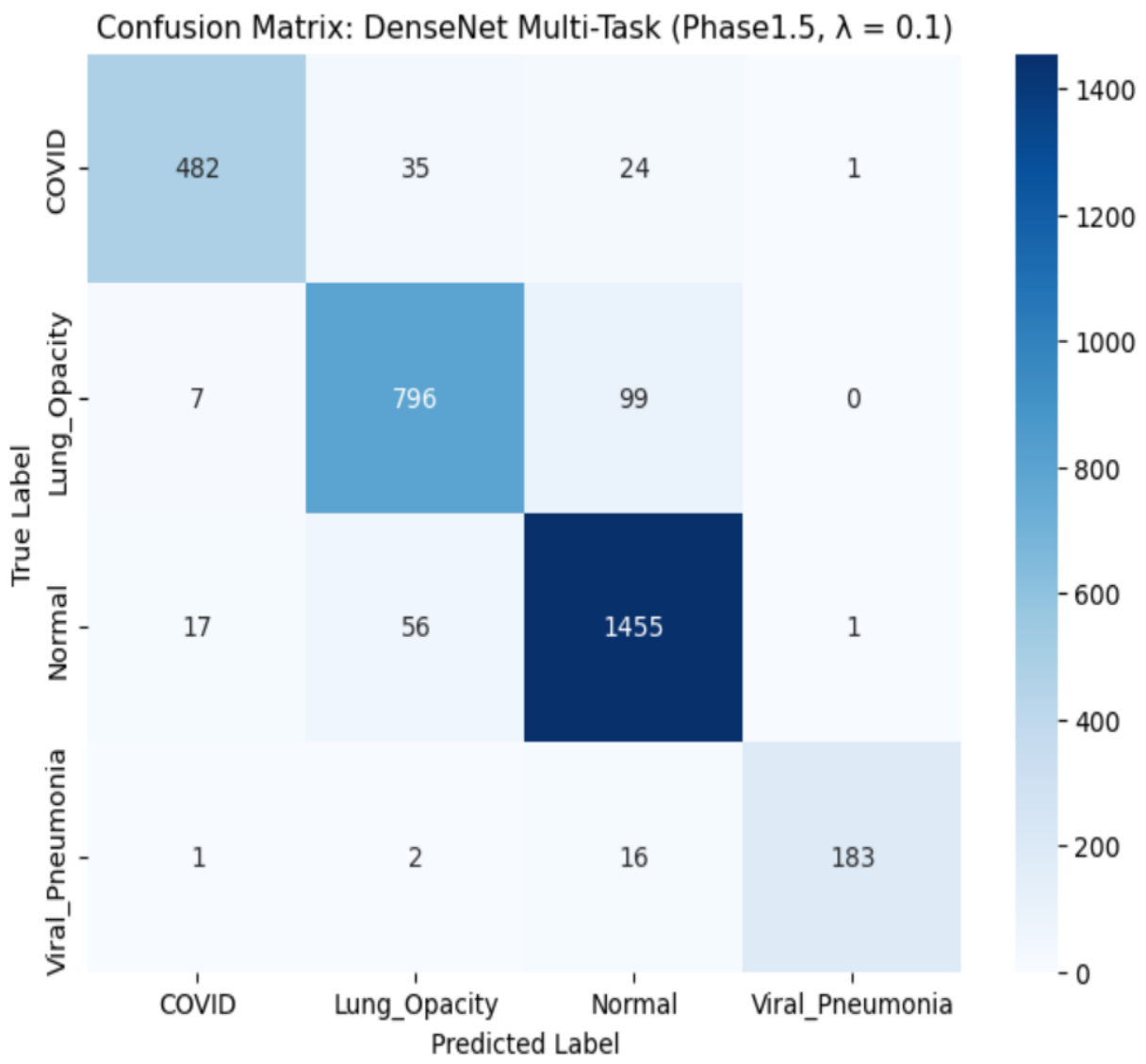


Key observations

1. High COVID-19 recall (0.89), which helps to reduce missed diagnoses
2. Normal class recall (0.95), meaning there are low false positives
3. Viral Pneumonia, highest performance (F1 = 0.95)
4. Slight confusion between COVID-19 & Lung Opacity

Confusion Matrix For Final Model (Phase 1.5)

Multi-Task DenseNet121



KEY OBSERVATIONS

Correct Classification

Strong correct classification across all four classes, with high true positives on the diagonal.

COVID-19 & Normal

High true positives for COVID-19 (482) and Normal (1,455), the two largest classes.

Main Confusion Zone

Most misclassifications occur between COVID-19 and Lung Opacity (35 cases). Both produce similar ground-glass opacities on CXR.

Viral Pneumonia

Minimal confusion with other classes (only 19 off-diagonal). Distinct radiographic pattern aids separation.

Supports Earlier Metrics

Distribution confirms the strong per-class precision and recall reported in Phase 1.5 results.

ANALYSIS

Segmentation Results & Grad-CAM Explainability

SEGMENTATION PERFORMANCE

1. High-quality lung boundary delineation confirmed by Dice and IoU metrics
2. Segmentation supervision guides the encoder toward anatomically meaningful features
3. Shared encoder benefits both tasks simultaneously

CLINICAL SIGNIFICANCE

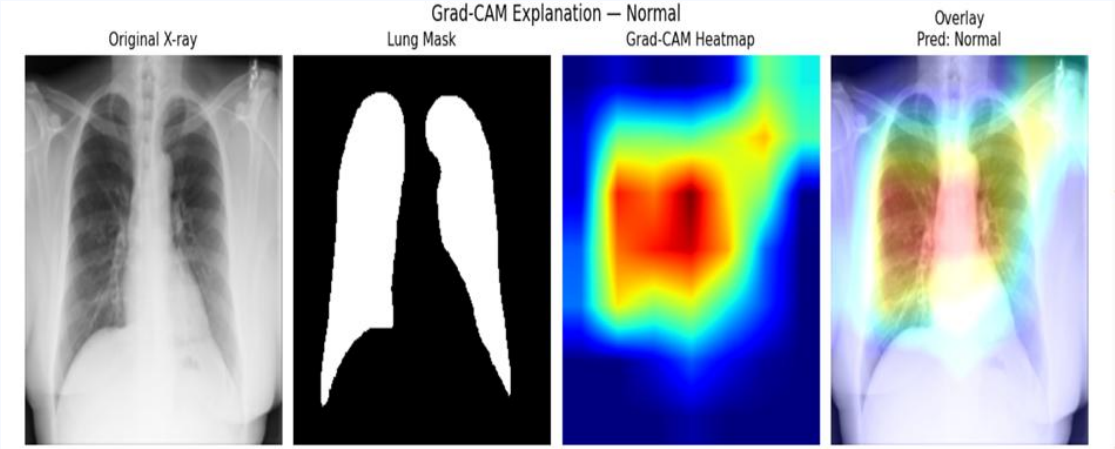
Lung masks provide anatomical guidance, allowing the model to focus on lung regions and reducing false activations from non-pulmonary structures (ribs, diaphragm).

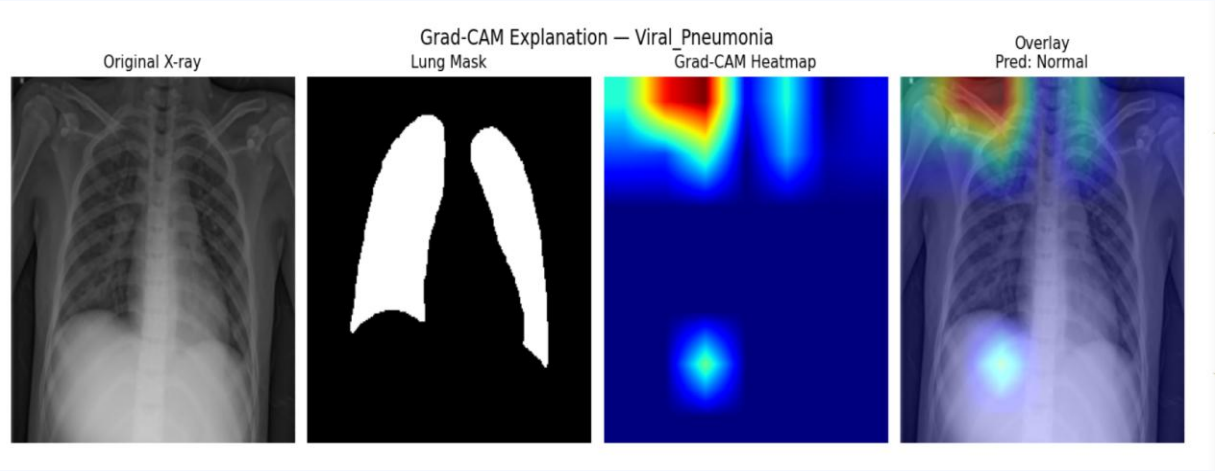
GRAD-CAM EXPLAINABILITY

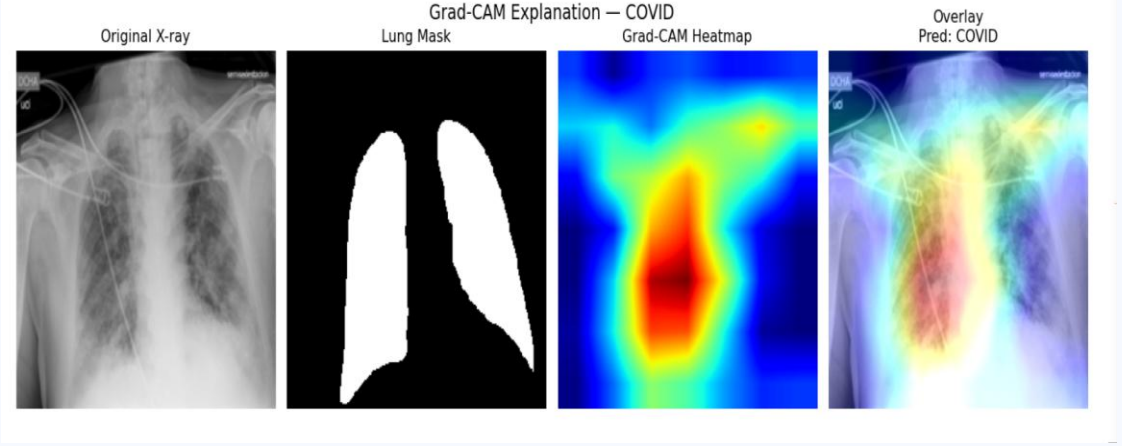
1. Extract gradients from last convolutional layer of DenseNet encoder
2. Compute weighted average of feature maps using gradient weights
3. Apply ReLU and upsample heatmap to match input image dimensions
4. Overlay heatmap on chest X-ray to highlight disease-relevant regions

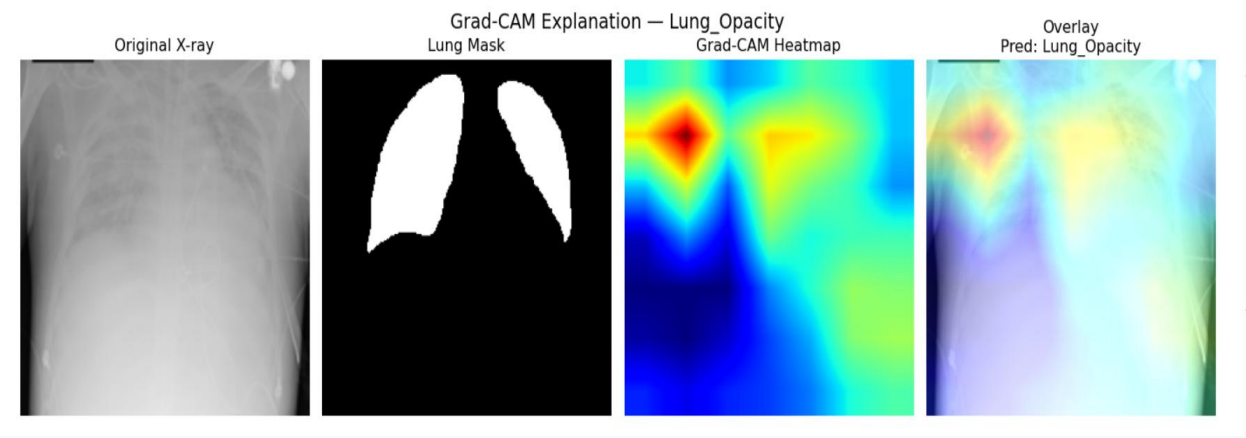
See Grad Cam images on the next slide…..

Grad CAM Images Output









BENCHMARKING

Metrics Comparison with Related Works and some Literature

Our Final Model — Phase 1.5

OUR MODEL — PER-CLASS PERFORMANCE (PHASE 1.5)

	COVID-19	LUNG OPACITY	NORMAL	VIRAL PNEUMONIA
Precision	0.95	0.90	0.91	0.99
Recall	0.89	0.88	0.95	0.91
F1-Score	0.92	0.89	0.93	0.95

OVERALL

Accuracy	0.92
AUC	0.986
Dice	0.89
IoU	0.80

RELATED WORKS COMPARISON

	Luján-García et al. (2020)	Nahiduzzaman et al. (2023)	Hertel & Benlamri (2021)	Shamrat et al. (2023)	OUR MODEL Phase 1.5
Accuracy	0.91	0.909		0.970	0.92
Precision	0.92			0.967	
Recall	0.82			0.968	
F1-Score	0.88				
AUC	0.98	0.969			0.986
Dice					0.89
IoU					0.80
COVID sens			≥ 0.95		
Models			3 & 2-class		
COVID acc		0.994			
	COVID class F1: 0.97	*4-class CXR model*	*COVID sensitivity ≥ 0.95*	*High-accuracy benchmark*	**Best AUC in class**

KEY INTERPRETATION

- ↑ Performance within the upper range of published multiclass CXR studies
- ◉ AUC 0.986 is competitive with the strongest published models
- ✓ Balanced precision & recall across all four disease classes
- + Segmentation adds anatomical guidance beyond classification-only studies

Limitations

1. Dataset imbalance was not explicitly handled during training, despite being a limitation, it did not materially affect performance
2. Kaggle dataset may not fully represent clinical conditions
3. Similarity between COVID-19 and Lung Opacity affects classification
4. No external validation dataset used

Final Model Summary - Metrics

DenseNet121

0.92

0.986

0.89

0.80

TensorFlow 2.x

Possible Future Work

- Apply class weighting or focal loss to address imbalance
- Use real clinical multi-center datasets
- Extend to multi-modal imaging (CT + X-ray)

Conclusion

1. Demonstrated that DenseNet121 significantly outperforms EfficientNet and ResNet on COVID-19 chest X-ray classification (0.82 vs. 0.48)
2. Achieved 0.89 accuracy through strategic fine-tuning, unfreezing the last 40 layers with a low learning rate (1e-5)
3. Developed a novel multi-task architecture achieving 0.92 accuracy and 0.986 AUC matching state-of-the-art performance
4. Added lung segmentation (Dice 0.89, IoU 0.80) providing anatomical context and improving classification robustness
5. Applied Grad-CAM visualisations to provide clinically interpretable explanations for model predictions

Final Model Summary - Metrics

DenseNet121
0.92
0.986
0.89
0.80
TensorFlow 2.x

References

1. Appasami, G., et al. (2022). Deep learning-based COVID-19 classification using chest X-ray images.

2. Chaddad, A., et al. (2021). Deep convolutional neural network models for predicting COVID-19 in CT and X-ray images.

3. Duong, L. T., Nguyen, P. T., Iovino, L., & Flammini, M. (2023). Automatic detection of COVID-19 from chest X-ray and CT images using deep neural networks and transfer learning. Applied Soft Computing, 132, 109851.

4. Khan, A. I., Shah, J. L., & Bhat, M. M. (2020). CoroNet: A deep neural network for detection and diagnosis of COVID-19 from chest X-ray images. Computer Methods and Programs in Biomedicine, 196, 105581. https://doi.org/10.1016/j.cmpb.2020.105581

5. Mahmud, T., Rahman, M. A., & Fattah, S. A. (2020). CovXNet: A multi-dilation convolutional neural network for automatic COVID-19 and other pneumonia detection from chest X-ray images. Computers in Biology and Medicine, 122, 103869. https://doi.org/10.1016/j.compbiomed.2020.103869

6. Masud, M., et al. (2022). A lightweight CNN architecture for COVID-19 detection using chest X-rays.

7. Nahiduzzaman, M., et al. (2023). Parallel CNN-ELM: A multiclass classification of chest X-ray images to identify multiple lung diseases. Expert Systems with Applications, 229, 120528.

8. Park, S., et al. (2022). Multi-task vision transformer using chest X-ray features for COVID-19 diagnosis and severity quantification. Medical Image Analysis, 75, 102299.

9. Phumkuea, T., Wongsirichot, T., Damkliang, K., Navasakulpong, A., & Andritsch, J. (2023). MSTAC: A multi-stage automated classification of COVID-19 chest X-ray images using stacked CNN models. Tomography, 9, 2233–2246.

10. Yen, C.-T., & Tsao, C.-Y. (2024). Lightweight convolutional neural network for chest X-ray image classification.